Project Report

Complaint/ Grievance Management System

Name: Anshika Patwari Riyansh Saxena	**SAP ID:** 70562300001 70562300095
Roll No. : A007, A024	**Class:** BTech (AI&DS)
Subject: Agile & Prod. Development	

Abstract

Efficient grievance management is essential for maintaining transparency and accountability in academic institutions. Traditional systems often suffer from delayed responses and lack of proper tracking mechanisms. This project presents the development of a web-based Grievance Management System using the PERN stack (PostgreSQL, Express.js, React.js, Node.js).

The system incorporates secure authentication, role-based access control, SLA-based resolution tracking, and real-time communication features, ensuring efficient handling of student complaints. Agile methodology was used for iterative development, while DevOps tools enabled automated testing, deployment, and monitoring.

The proposed solution improves grievance resolution efficiency, enhances transparency, and provides a scalable and user-friendly platform for both students and administrators.

1. Introduction

With the rapid advancement of digital technologies, educational institutions are increasingly adopting software solutions to streamline administrative processes. One critical area that requires efficient management is the handling of student grievances. Traditional grievance systems, which often rely on manual processes or basic online forms, lack transparency, structured tracking, and timely resolution mechanisms.

In many cases, students are unable to track the progress of their complaints, and authorities face challenges in managing and prioritizing multiple grievances effectively. This results in delays, miscommunication, and reduced trust in the system.

To address these challenges, the **NMIMS Grievance Management System** has been developed as a centralized, web-based platform that enables efficient handling of complaints. The system is designed to manage the complete lifecycle of a grievance, from submission to resolution, while ensuring transparency and accountability at every stage.

The application is built using the **PERN stack**, providing a robust and scalable architecture. It incorporates modern features such as **secure authentication, role-based access control, SLA-driven resolution tracking, and real-time communication through an integrated chat system**.

Furthermore, the project follows **Agile development practices**, allowing iterative improvements and continuous feedback. The integration of **DevOps tools** ensures reliable deployment, automated testing, and system monitoring.

Overall, this system aims to transform the traditional grievance handling process into a **structured, efficient, and user-friendly digital solution** for both students and administrative authorities.

2. Objectives

The primary objective of this project is to design and develop a **robust, scalable, and efficient grievance management system** for academic institutions. The system aims to improve the overall process of handling student complaints by introducing structure, transparency, and automation.

The specific objectives of the project are as follows:

- To develop a **centralized web-based platform** for submitting and managing grievances
- To implement **secure user authentication** using JSON Web Tokens (JWT)
- To design a **role-based access control system (RBAC)** for students, authorities, and administrators
- To enable **real-time tracking** of grievance status throughout its lifecycle
- To incorporate an **SLA-based resolution mechanism** for timely handling of complaints
- To provide a **communication system** between users and authorities through an integrated chat feature
- To maintain **audit logs** for tracking all system activities and ensuring accountability
- To build a **responsive and user-friendly interface** using modern frontend technologies
- To integrate **DevOps tools** for automated testing, deployment, and monitoring
- To ensure the system is **scalable, reliable, and suitable for real-world deployment**

These objectives collectively aim to transform the traditional grievance handling approach into a **modern, efficient, and transparent digital system**.

3. Methodology (Agile – Scrum Framework)

The development of the NMIMS Grievance Management System was carried out using the **Agile methodology**, specifically the **Scrum framework**. This approach was chosen to ensure flexibility, faster development, and continuous improvement throughout the project lifecycle.

3.1 Agile Methodology

Agile is an iterative and incremental approach to software development that focuses on delivering small, functional components of the system in multiple stages. It emphasizes:

- Continuous feedback
- Rapid delivery of features
- Adaptability to changing requirements

In this project, Agile enabled the team to progressively enhance the system while incorporating feedback at each stage.

3.2 Scrum Framework

Scrum is a structured Agile framework that divides the project into fixed time periods called **Sprints**. Each sprint focuses on delivering specific features of the system.

Key components of Scrum used in this project include:

- **Sprint Planning**: Identifying tasks and goals for each sprint
- **Incremental Development**: Building the system in phases
- **Continuous Testing and Feedback**: Ensuring quality at every stage

3.3 Sprint-wise Implementation

The project was divided into four major sprints, each focusing on a specific aspect of the system:

Sprint 1: Security and Backend Foundation

- Designed the database schema using PostgreSQL
- Implemented backend APIs using Node.js and Express.js
- Developed secure authentication using JWT

Sprint 2: Frontend Development

- Built the user interface using React.js
- Implemented responsive design with Tailwind CSS
- Designed dashboards and forms for user interaction

Sprint 3: Core Functional Features

- Implemented role-based access control (RBAC)
- Developed the SLA engine for grievance prioritization
- Created admin functionalities for managing complaints

Sprint 4: Communication and Testing

- Developed the chat/timeline system for communication
- Performed automated testing using Selenium
- Conducted debugging and final optimization

3.4 Advantages of Using Agile in This Project

- Allowed continuous improvement of features
- Reduced development risks through iterative testing
- Enabled faster identification and resolution of issues
- Improved overall system quality and usability

The use of Agile and Scrum ensured that the project was developed in a **structured, flexible, and efficient manner**, resulting in a high-quality final product.

4. System Architecture (PERN Stack)

The NMIMS Grievance Management System is designed using a **full-stack PERN architecture**, which ensures high performance, scalability, and maintainability. The PERN stack consists of **PostgreSQL, Express.js, React.js, and Node.js**, each responsible for a specific layer of the application.

4.1 Overview of PERN Stack

The PERN stack is a modern web development architecture where all components are based on JavaScript (except PostgreSQL), enabling seamless communication between the frontend and backend.

- **PostgreSQL** → Database layer
- **Express.js** → Backend framework
- **React.js** → Frontend library
- **Node.js** → Runtime environment

This architecture allows efficient handling of client requests, data processing, and user interface rendering.

4.2 Frontend Layer – React.js

What it is:
React.js is a JavaScript library used to build interactive and dynamic user interfaces, particularly for Single Page Applications (SPA).

Use in the Project:

- Developed a responsive and interactive user interface
- Implemented dashboards for students and administrators
- Managed components such as grievance forms, chat system, and status tracking
- Enabled real-time updates without page reloads

The frontend ensures a smooth user experience with fast rendering and intuitive navigation.

4.3 Backend Layer – Node.js and Express.js

What is Node.js:
Node.js is a runtime environment that allows JavaScript to be executed on the server side.

What is Express.js:
Express.js is a lightweight web framework used for building APIs and handling server-side logic.

Use in the Project:

- Developed RESTful APIs for communication between frontend and database
- Handled request processing and response generation
- Implemented middleware for authentication and authorization
- Managed business logic such as grievance submission and status updates

The backend acts as the core processing unit of the system.

4.4 Database Layer – PostgreSQL

What it is:
PostgreSQL is a powerful open-source relational database management system.

Use in the Project:

- Stored all application data including users, grievances, comments, and logs
- Maintained structured and relational data
- Ensured data consistency and integrity
- Supported complex queries for efficient data retrieval

4.5 ORM Layer – Prisma

What it is:
Prisma is an Object Relational Mapping (ORM) tool that simplifies database interactions using code.

Use in the Project:

- Provided a type-safe interface for database queries
- Simplified CRUD (Create, Read, Update, Delete) operations
- Managed database schema and migrations
- Reduced complexity in handling SQL queries

4.6 System Workflow

The working of the system follows a structured flow:

1. The user interacts with the **frontend interface (React.js)**
2. Requests are sent to the **backend server (Node.js + Express.js)**
3. The backend processes the request and communicates with the **PostgreSQL database via Prisma ORM**
4. The database returns the required data
5. The backend sends a response back to the frontend
6. The frontend updates the UI accordingly

4.7 Advantages of PERN Architecture

- High performance and fast data processing
- Scalability for handling large user bases
- Modular and maintainable code structure
- Seamless integration between frontend and backend

The use of the PERN stack ensures that the system is **efficient, scalable, and suitable for real-world deployment**, making it a strong foundation for the grievance management platform.

5. Core Features of the System

The NMIMS Grievance Management System incorporates several key features designed to ensure efficient handling of grievances, improved transparency, and enhanced user experience. Each feature is implemented with a specific purpose to address the limitations of traditional grievance systems.

5.1 User Authentication (JWT-Based)

What it is:
Authentication is the process of verifying the identity of a user. The system uses **JSON Web Tokens (JWT)** to securely manage user sessions.

Use in the Project:

- Users must log in to access the system
- JWT tokens are generated upon successful login
- Tokens are used to verify user identity for each request
- Passwords are securely stored using encryption

Importance:

- Ensures system security
- Prevents unauthorized access
- Protects sensitive user data

5.2 Role-Based Access Control (RBAC)

What it is:
RBAC is a mechanism that restricts system access based on user roles.

Use in the Project:
The system defines three main roles:

- **Student/User** → Can submit and track grievances
- **Authority** → Can view and resolve assigned grievances
- **Admin** → Has full control over the system

Each role has different permissions, ensuring controlled access to system features.

Importance:

- Enhances system security
- Prevents misuse of administrative functions
- Maintains structured workflow

5.3 Grievance Lifecycle Management

What it is:
This feature manages the complete process of a grievance from submission to resolution.

Use in the Project:

- Students submit grievances through a form
- Authorities review and update status
- Status changes include Pending, In Progress, Resolved
- Users can track the progress in real time

Importance:

- Provides structured complaint handling
- Ensures transparency
- Improves user trust

5.4 SLA Engine (Service Level Agreement)

What it is:
An SLA defines the expected time within which a grievance should be resolved.

Use in the Project:

- Each grievance is assigned a priority level
- Based on priority, a resolution deadline is automatically generated
- Authorities are expected to resolve issues within the defined time

Importance:

- Ensures timely resolution
- Helps monitor delays
- Improves accountability

5.5 Real-Time Chat / Timeline System

What it is:
A communication feature integrated within each grievance.

Use in the Project:

- Enables interaction between students and authorities
- Maintains a conversation timeline
- Allows users to provide updates or additional details

Importance:

- Improves communication clarity
- Reduces misunderstandings
- Enhances user engagement

5.6 Audit Logs

What it is:
Audit logs record every activity performed within the system.

Use in the Project:

- Tracks actions such as:
    - Status updates
    - Priority changes
    - Comments
- Stores logs permanently in the database

Importance:

- Ensures accountability
- Helps in tracking system activity
- Useful for analysis and reporting

5.7 User Interface and Experience (UI/UX)

What it is:
The design and usability of the application interface.

Use in the Project:

- Developed using React.js and Tailwind CSS
- Implemented a modern glassmorphic design
- Designed responsive layouts for different devices
- Included interactive elements for better usability

Importance:

- Enhances user satisfaction
- Makes the system easy to use
- Improves accessibility and engagement

5.8 API Integration (Axios)

What it is:
Axios is a library used to send HTTP requests from frontend to backend.

Use in the Project:

- Handles communication between frontend and backend
- Sends data such as login credentials and grievance details
- Receives responses from the server

Importance:

- Enables smooth data exchange
- Ensures real-time updates
- Improves system responsiveness

Overall, these features work together to create a **comprehensive, efficient, and user-friendly grievance management system**, addressing both user needs and administrative requirements.

6. DevOps and Tools Implementation

To ensure that the NMIMS Grievance Management System is reliable, scalable, and production-ready, modern **DevOps practices** were incorporated into the development process. DevOps focuses on automating and streamlining the stages of software development, testing, deployment, and monitoring.

6.1 DevOps Overview

What is DevOps:
DevOps is a combination of development (Dev) and operations (Ops) practices that aims to shorten the software development lifecycle and deliver high-quality applications continuously.

Use in the Project:

- Automated build and deployment processes
- Continuous testing and integration
- Improved collaboration between development and deployment stages

6.2 Git (Version Control System)

What it is:
Git is a distributed version control system used to track changes in source code.

Use in the Project:

- Managed all project files and source code
- Implemented a **feature-branch workflow**

- Ensured safe integration of new features into the main branch
- Maintained version history for debugging and rollback

Importance:

- Enables collaboration
- Prevents code conflicts
- Provides traceability of changes

6.3 Docker (Containerization)

What it is:
Docker is a platform used to package applications and their dependencies into containers.

Use in the Project:

- Containerized the PostgreSQL database
- Ensured consistent development and deployment environments
- Simplified system setup across different machines

Importance:

- Eliminates environment-related issues
- Improves portability
- Speeds up deployment

6.4 Jenkins (Continuous Integration and Deployment)

What it is:
Jenkins is an automation tool used for Continuous Integration (CI) and Continuous Deployment (CD).

Use in the Project:

- Automatically triggered build process on code updates
- Executed test cases before deployment
- Ensured only stable code is deployed

Importance:

- Reduces manual effort
- Detects errors early
- Improves software quality

6.5 Kubernetes (Container Orchestration)

What it is:
Kubernetes is a platform used to manage and orchestrate containerized applications.

Use in the Project:

- Managed deployment of application containers
- Enabled scaling based on user demand
- Automatically restarted failed components

Importance:

- Ensures high availability
- Supports scalability
- Improves system reliability

6.6 Selenium (Automated Testing)

What it is:
Selenium is a tool used for automating web application testing.

Use in the Project:

- Simulated real user actions such as login and grievance submission
- Tested end-to-end functionality
- Ensured consistent performance after updates

Importance:

- Reduces manual testing effort
- Improves accuracy
- Ensures bug-free application

6.7 Monitoring Tools (Prometheus and Grafana)

What they are:

- **Prometheus** → Collects system metrics
- **Grafana** → Visualizes data through dashboards

Use in the Project:

- Monitored API performance and response time
- Tracked SLA compliance
- Detected system issues in real time

Importance:

- Helps maintain system health
- Enables proactive issue resolution
- Provides performance insights

6.8 Benefits of DevOps Integration

The integration of DevOps practices in this project resulted in:

- Faster and more reliable deployments
- Improved system stability
- Early detection of errors
- Continuous improvement of the application

Overall, the use of DevOps tools ensured that the system is **scalable, efficient, and suitable for real-world deployment environments**.

7. Results and Discussion

The implementation of the NMIMS Grievance Management System demonstrates a significant improvement over traditional grievance handling methods. The system successfully achieves its objective of providing a **structured, transparent, and efficient platform** for managing student complaints.

7.1 Results

The developed system delivered the following outcomes:

- **Efficient Grievance Handling**: Complaints are processed in a structured manner from submission to resolution
- **Real-Time Tracking**: Users can monitor the status of their grievances at every stage
- **Improved Communication**: The integrated chat system enables direct interaction between students and authorities
- **Enhanced Security**: JWT-based authentication and RBAC ensure secure system access
- **Accountability**: Audit logs provide a complete record of all system activities
- **Timely Resolution**: SLA-based deadlines help ensure grievances are addressed within defined timeframes

7.2 Discussion

The system addresses key limitations of traditional grievance mechanisms by introducing automation and transparency. The use of modern technologies and structured workflows ensures that both students and administrators benefit from a more organized process.

The integration of DevOps tools further enhances the system by:

- Ensuring reliable deployment
- Reducing system downtime
- Improving overall performance and stability

Additionally, the use of Agile methodology allowed continuous refinement of features, resulting in a more user-centric and efficient application.

However, the system can be further improved by incorporating advanced features such as AI-based categorization and predictive analytics for better decision-making.

8. Future Scope

While the current system provides a robust and efficient solution for grievance management, there are several enhancements that can be implemented in the future to further improve its functionality and scalability.

- **AI-Based Grievance Categorization**:
  Integration of Natural Language Processing (NLP) to automatically categorize grievances based on their content, reducing manual effort and improving accuracy.
- **Predictive Analytics**:
  Use of historical data to predict trends in grievances, helping authorities identify recurring issues and take preventive measures.
- **Mobile Application Development**:
  Development of a mobile application using frameworks such as React Native to increase accessibility for users.
- **Integration with Institutional Systems**:
  Linking the portal with existing university ERP systems for seamless data sharing and centralized management.
- **Notification System**:
  Implementation of email and SMS notifications to keep users updated about grievance status changes.
- **Advanced Reporting Dashboard**:
  Providing detailed analytics and reports for administrators to monitor performance and decision-making.

9. Conclusion

The NMIMS Grievance Management System successfully demonstrates the application of modern software engineering principles in addressing real-world administrative challenges.

The system provides a **secure, scalable, and user-friendly platform** that improves the efficiency of grievance handling while ensuring transparency and accountability. The integration of features such as **role-based access control, SLA-driven resolution, real-time communication, and audit logging** enhances both user experience and administrative control.

Furthermore, the adoption of **Agile methodology** enabled systematic and iterative development, while the use of **DevOps tools** ensured reliable deployment, testing, and monitoring of the application.

Overall, the project transforms the traditional grievance management process into a **structured and technology-driven solution**, making it suitable for deployment in modern educational institutions.

10. Pictorial Representation

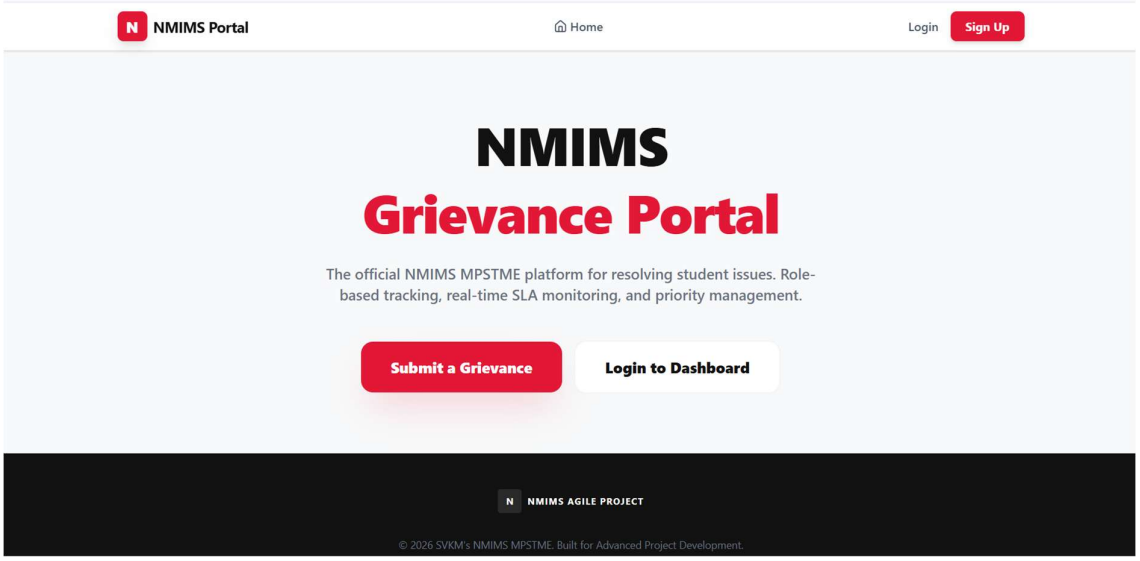

Fig.1. Application Home page

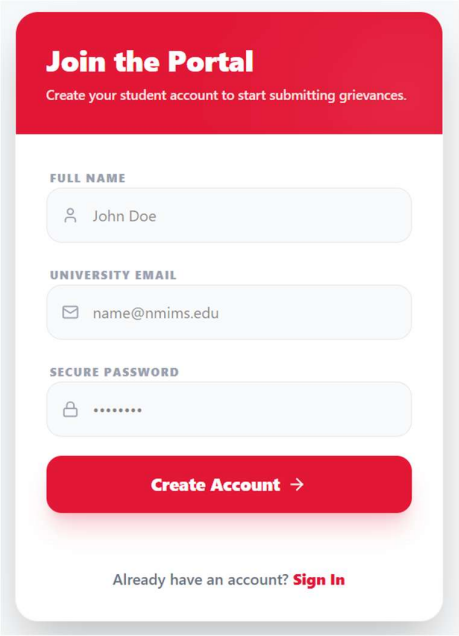

Fig. 2 Sign-Up Page

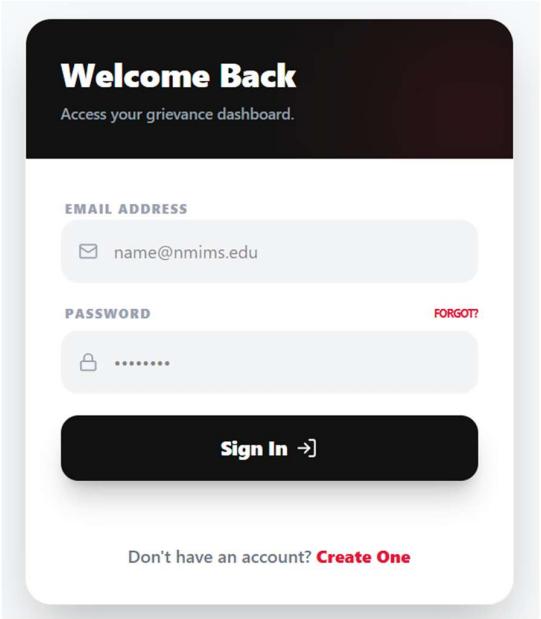

Fig.3 Login Page

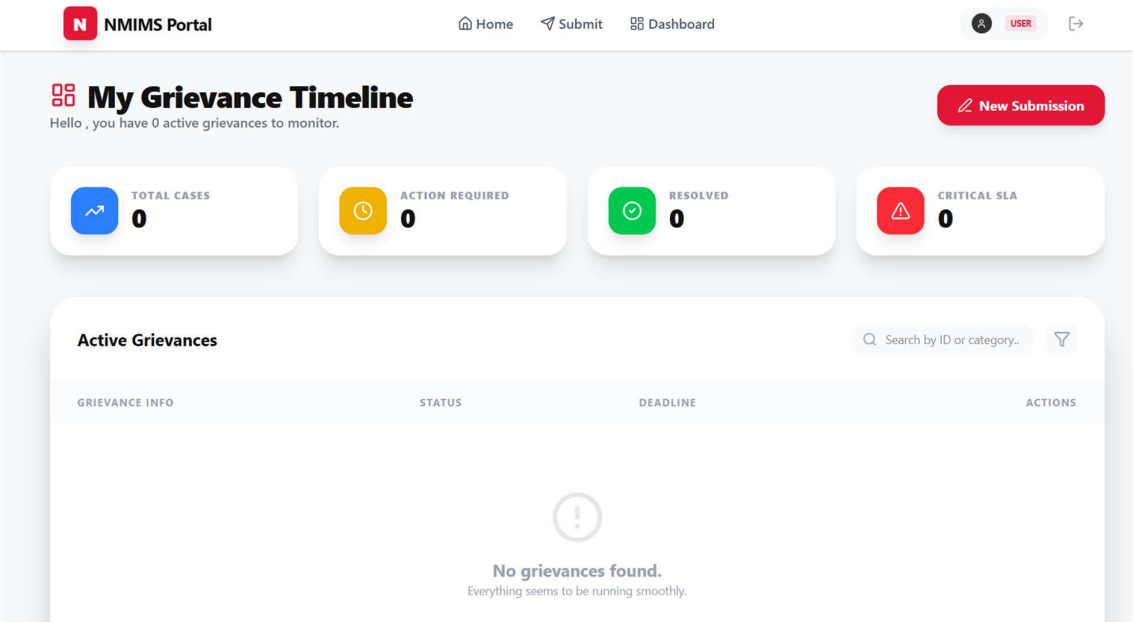

Fig. 4 Student Grievance Portal

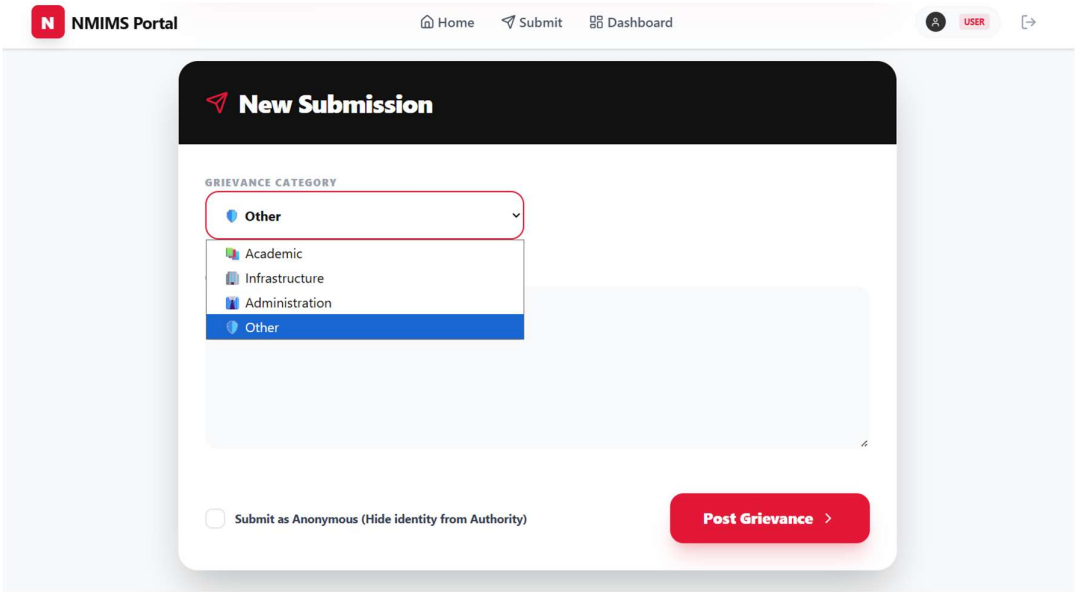

Fig. 5 New Grievance/Complaint Submission

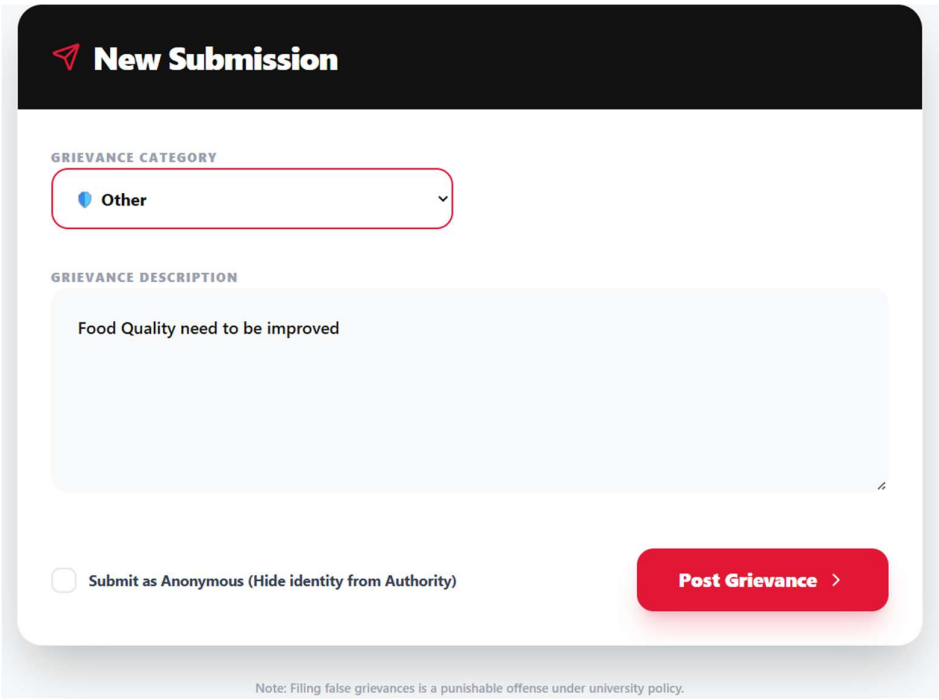

Fig. 6 New Grievance/Complaint Submission

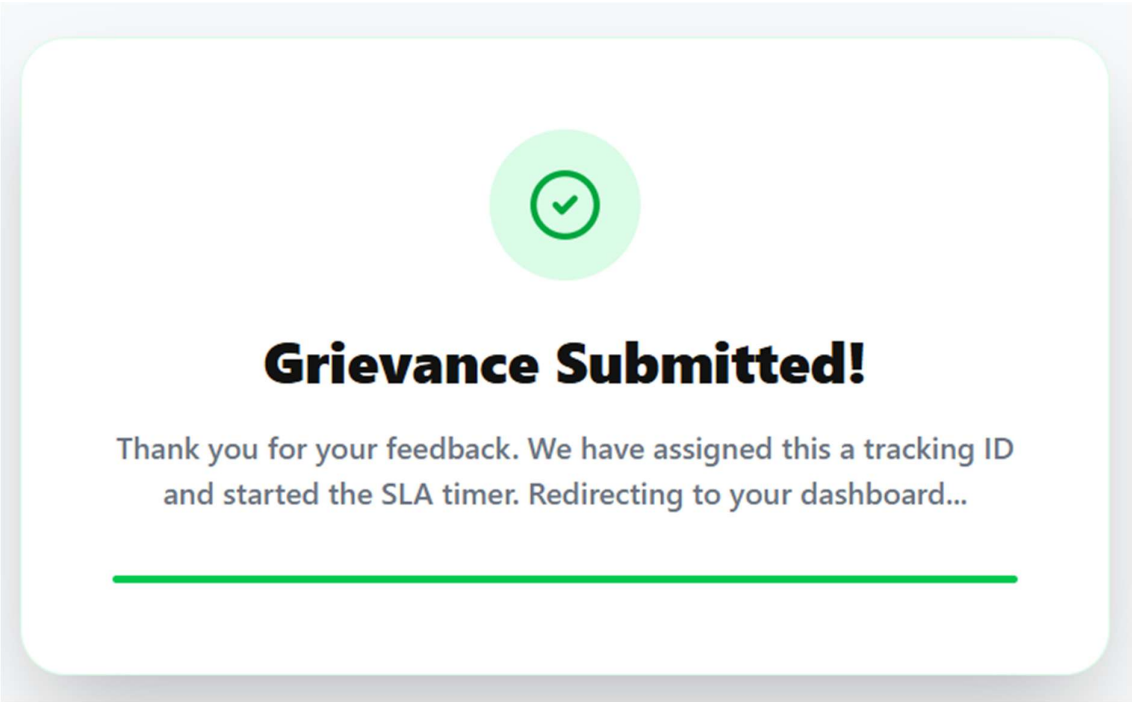

Fig. 7 Grievance Submitted

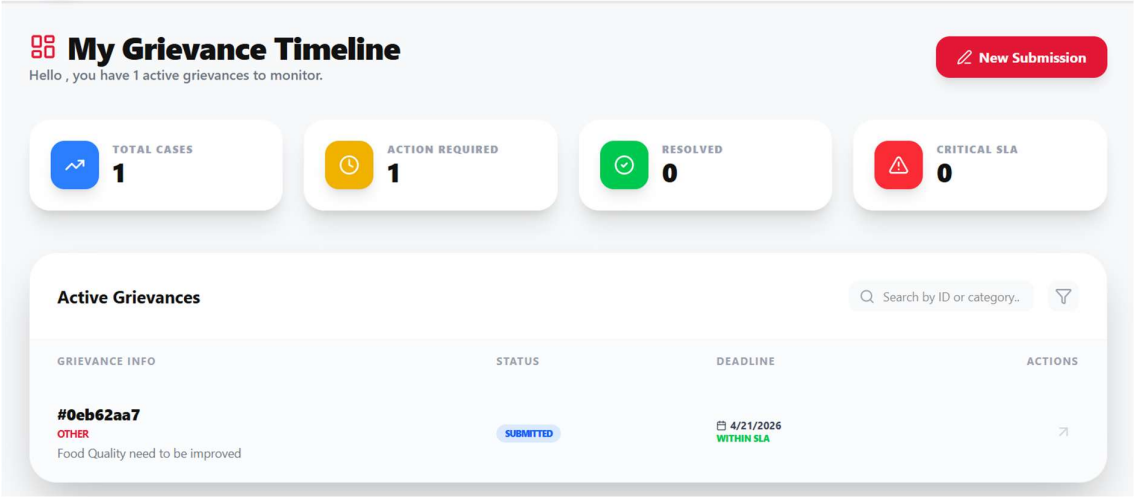

Fig.8 Updated in Student Portal

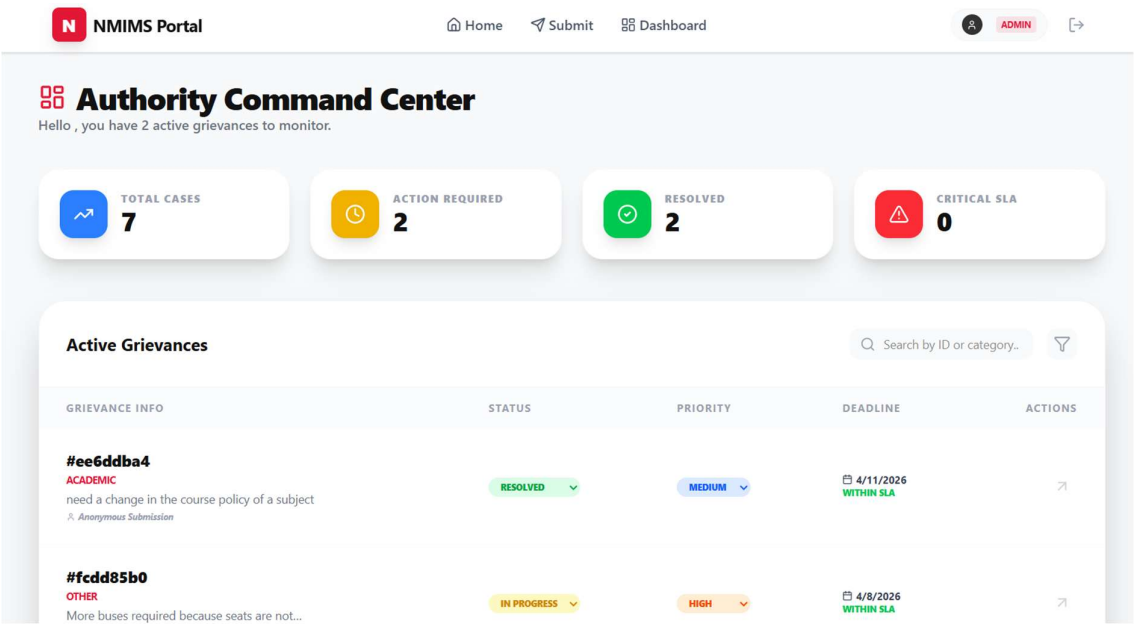

Fig. 9 Admin/Authorities Access Portal

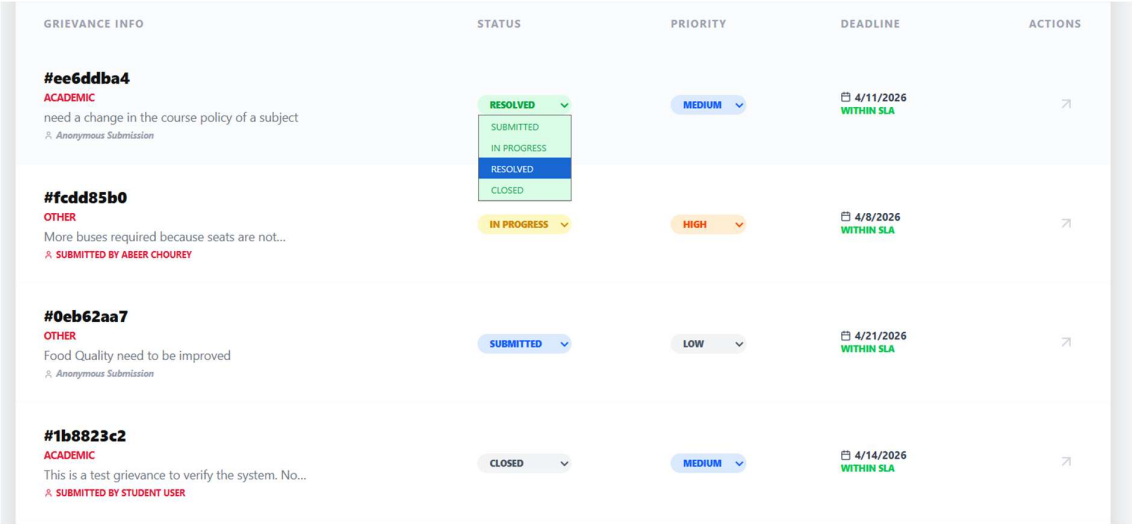

Fig. 10 Complaints Received

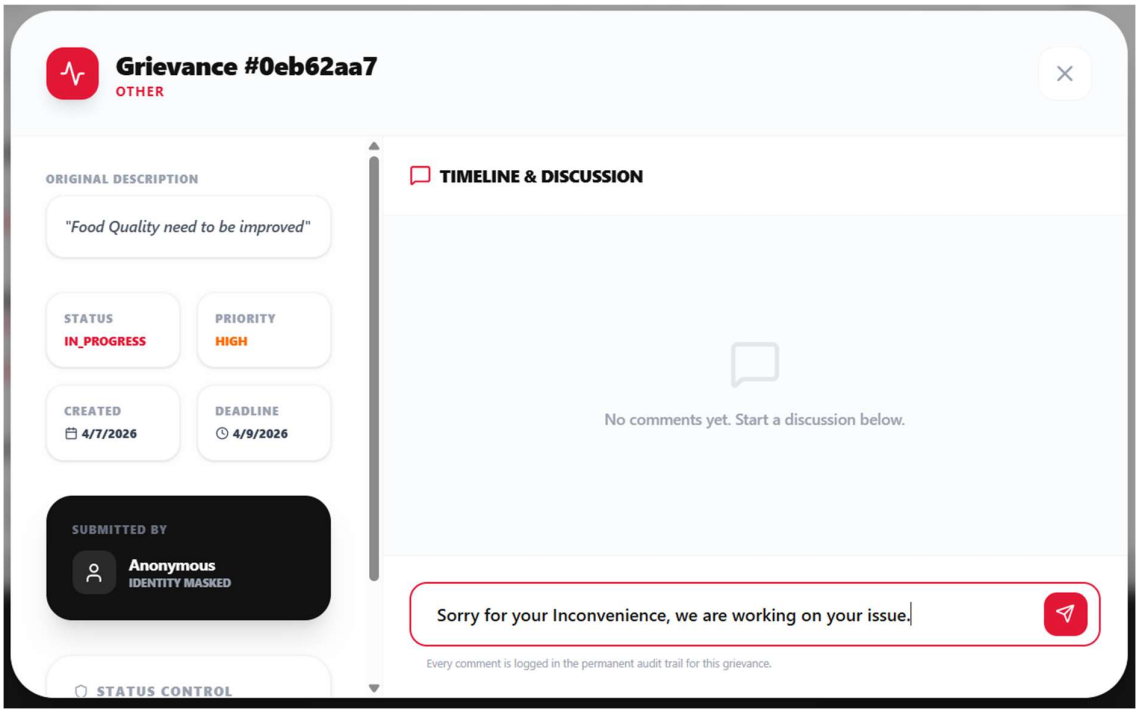

Fig.11 Comment/Chat section between Student and Admin